

FlyArm: A Frozen Whole-CNS Fruit-Fly Connectome as the Controller of a Robot Arm

Technical report

FlyArm project

<https://github.com/yusenthebot/FlyArm>

September 2026

Abstract

We use the complete connectome of the adult male *Drosophila* central nervous system (MaleCNS v1.0: 166,700 neurons, 10.5 million connections) as the controller of a simulated Franka arm and never train it. Only the interface is trained: a sensory encoder that writes currents into the 1,846 ascending neurons and a linear decoder that reads the 1,314 descending and 708 VNC motor neurons into the action; every signal from observation to action passes through three steps of a rate model on the measured wiring per control step. With imitation followed by PPO with a demonstration term, this controller solves all four tasks of D4RL FrankaKitchen in 50 of 50 test episodes of the official environment, including starts it was never trained on. On a new articulated multi-step manipulation benchmark (drawers, a lidded cabinet with a shelf, a bin, stacking; tasks of 3 to 8 sub-goals over up to 2,600 control steps), skill-level DAGger followed by PPO that also trains the encoder through one step of the frozen connectome reaches 69.6% full-task success on held-in test episodes and 75.0% with objects never seen in training, against 51.3% and 49.6% for imitation alone. Held-out furniture and task orders remain hard (36.6% and 26.6%). We describe the method, the benchmark, what did and did not work, and the controls that are still to run: this report shows that a frozen whole-CNS connectome can be trained to control the arm, not that its wiring is what makes this possible.

1 Introduction

Whole-brain connectomes of the fruit fly now exist for the adult brain [2] and for the complete male central nervous system, brain and ventral nerve cord together [1]. Connectome-constrained networks predict neural activity in the fly visual system when their weights are fixed by the wiring and only a few parameters are fitted [6], and biomechanical fly models close sensorimotor loops with connectome-constrained circuits [13]. We ask a different, engineering question: can the complete measured wiring of an insect nervous system, frozen as it is, serve

as the controller of a robot that has to solve long-horizon manipulation tasks?

FlyArm makes the connectome the only path from perception to action. Observations enter as currents into the neurons annotated as ascending; actions are read from the neurons annotated as descending and VNC motor; the recurrent weights are the measured synapse counts with transmitter signs and are never updated. What is trained is the interface: an encoder on the input side and a decoder on the output side, as in reservoir computing [11], but with a reservoir that is a real nervous system of 166,700 neurons rather than a random network.

Contributions:

- A controller in which the complete MaleCNS connectome is frozen and the only trained parts are the encoder into its ascending neurons and the decoder out of its descending and motor neurons, running in real time on a laptop GPU (Section 2).
- A training recipe that makes this controller work on long-horizon tasks: skill-level DAGger from sub-goal resets, then PPO with a demonstration term in which the encoder is also updated through one control step of the frozen connectome with a clipped surrogate (Section 3).
- FrankaKitchen solved in the official environment, and an articulated multi-step manipulation benchmark with five preregistered splits on which the recipe reaches 70 to 75% full-task success on seen furniture (Section 5).
- A record of the negative results and failure modes, and an explicit account of which controls are still missing (Section 6).

2 Controller

Connectome. We compile the MaleCNS v1.0 release into a sparse matrix over its 166,700 annotated neurons. A connection is kept when two neurons share at least three synaptic contacts (10,520,377 pairs, 104 million

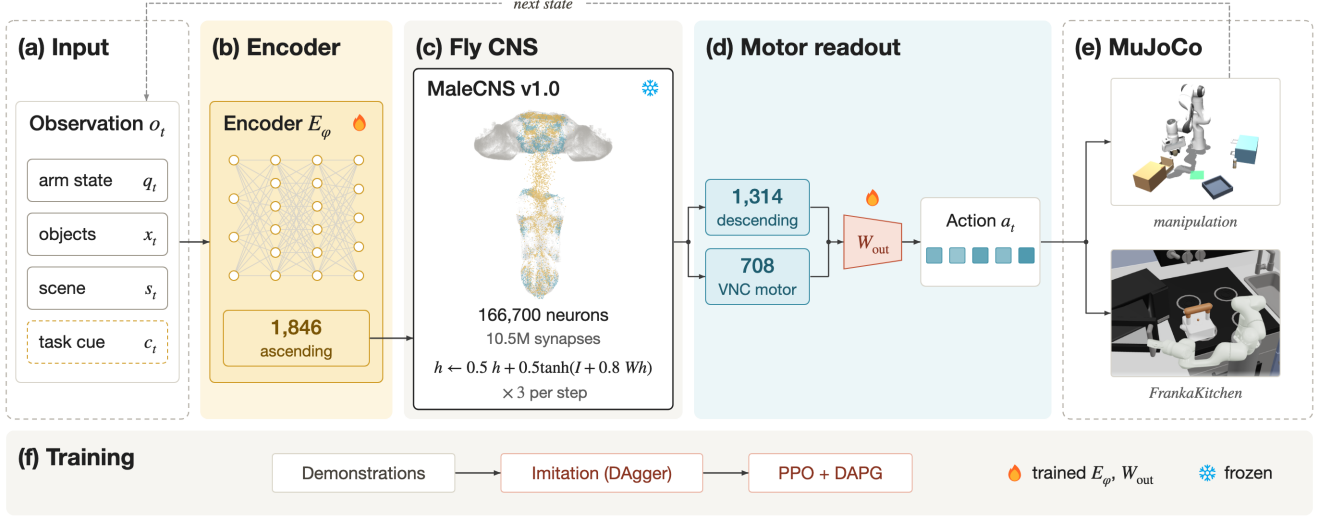


Figure 1: One control step of FlyArm. (a) The observation is simulator state: arm q_t , objects x_t , scene s_t and, on the manipulation benchmark, a task cue c_t . (b) The trained encoder E_ϕ writes currents into the 1,846 ascending neurons only. (c) The MaleCNS v1.0 connectome is frozen; each control step runs three steps of $h \leftarrow 0.5h + 0.5 \tanh(I + 0.8Wh)$ on all 166,700 neurons. The soma map shows measured soma locations (gray), the declared inputs (amber) and outputs (teal). (d) The 1,314 descending and 708 VNC motor neurons pass a fixed readout normalization and the trained linear decoder W_{out} into the action a_t . (e) MuJoCo executes the action; the renders are a frame of the reported checkpoint on a test episode and the FrankaKitchen scene. (f) Only E_ϕ and W_{out} are trained, by imitation and then PPO with DAPG.

synapses, 84% of all synapses between annotated neurons); its sign is +1 for acetylcholine, -1 for GABA and glutamate and 0 for other or unclear transmitters, and each neuron’s incoming absolute strength is normalized to at most one. The pack is hash-pinned and a run refuses any other pack.

Dynamics. The network state $h \in \mathbb{R}^{166,700}$ follows a leaky rate model,

$$h \leftarrow \frac{1}{2}h + \frac{1}{2} \tanh(I + gWh), \quad g = 0.8, \quad (1)$$

with three neural steps per control step at 20Hz; the readout is the mean of the output rates over the three steps, and the state persists across control steps within an episode. The recurrence runs in MLX [5] with a custom Metal sparse kernel, about 4 ms per control step on an Apple laptop GPU.

Interface. Input and output neurons are chosen by the official `superclass` annotation, not by hand: the input current I is non-zero only on the 1,846 `ascending_neuron` cells, and the decoder reads only the 1,314 `descending_neuron` and 708 `vnc_motor` cells. The two sets are disjoint, so with every connectome edge removed the output is exactly zero for any input: the connectome is the only channel from observation to action. On the manipulation benchmark the encoder E_ϕ is a two-layer tanh MLP of width 256 from the 300 observation features (below) to the 1,846 input currents; on

FrankaKitchen it is linear. Before the decoder, output rates are centred and scaled to unit expected norm with constants computed once from demonstrations and then frozen; without this, the first Adam updates on 2,022 correlated outputs saturate the decoder. The decoder W_{out} is linear with a tanh. On the manipulation benchmark the controller has 0.63 M trainable parameters (0.62 M in the encoder, 10 k in the decoder); the connectome has 10.5 M fixed weights.

Observation and action. On the manipulation benchmark the observation is 230 values of simulator state: the arm (21), four object slots of 22 values (position, offset from the hand, rotation, shape descriptor, grasp flag), the furniture (46) and a memoryless task cue (75) that names the current subgoal’s skill, object and receptacle, the relevant offsets, and a 10-way motor phase computed from geometric predicates of the scene (aligned, at height, grasped, above the target). Joint and object velocities are zeroed, because controllers cloned from states with velocities learned to keep doing what the velocities say. Seventy fixed copies of the hand-relative offsets and heading errors, $\tanh(\delta/\sigma)$ at σ of one and four control steps, let the encoder resolve the millimetres that decide between hovering and descending. The action is an end-effector step $(\Delta x, \Delta y, \Delta z, \Delta \psi, g)$ in $[-1, 1]$, at most 14 mm and 0.05 rad per step, executed by damped least-squares inverse kinematics on the Menagerie Panda [15] in MuJoCo [12] over 25 physics

steps of 2 ms. On FrankaKitchen the observation is the benchmark’s 30 position features and the action its 9 joint-velocity commands.

3 Training

Imitation. The teacher on the manipulation benchmark is a scripted controller with privileged state that re-derives its phase from the scene whenever the current subgoal changes, so it can label any state a learner reaches. Plain behaviour cloning and task-level DAgger [8] both failed on full tasks: the learner stalled early and never produced the states of later subgoals. Skill-level DAgger (Figure 2a) instead resets episodes to recorded starts of individual subgoals, so every skill is practised from its own start distribution from the first round. Each round collects 256 episodes, labels every state with the teacher, and trains on the aggregate with an L1 loss over windows of 16 control steps after a burn-in of 16 steps of neural state, with skill-balanced weights capped at five. After ten rounds the aggregate holds 1.8 M labelled steps; the checkpoint is selected on train-split validation episodes. On FrankaKitchen, imitation is DAgger with a tracker of the benchmark’s demonstrations.

PPO with a demonstration term. We fine-tune with PPO [10] and GAE [9] ($\gamma = 0.995$, $\lambda = 0.95$, clip 0.2) on 128 batched environments with rollouts of 64 steps, a critic on privileged state that is warmed up for 50 iterations before the policy moves, and advantages clipped at ten standard deviations. Every policy update adds the DAPG term [7], the squared error to the teacher’s action on a minibatch of 1,024 demonstration steps, with weight one. The reward pays a bonus of 50 per subgoal in task order, potential-based shaping toward the current subgoal (weight 10), a potential on the displacement of objects the task does not involve (weight 5), and small penalties for stray robot contact and large actions; every level term is a penalty, so stalling is never worth more than finishing. Training runs on a curriculum of two to three subgoals from subgoal resets and then full tasks, with subgoal resets drawn from a bank recorded by the teacher.

Training the encoder with PPO. With the encoder frozen, PPO tunes only the 10 k decoder weights. We also update the encoder, with the gradient truncated to one control step: the connectome state before the step is a constant, and only that step’s three neural updates are differentiated, so a single step’s graph is alive at a time. The encoder’s loss is PPO’s clipped surrogate on the rollout’s log-probabilities plus its own DAPG term, differentiated through the same single step from connectome states stored before 1,024 demonstration steps. Every 25 iterations the demonstrations are replayed through the current encoder, so the decoder’s DAPG features and the

stored states follow the encoder. An unclipped encoder update collapsed the policy twice in one run (validation 41 of 56 at iteration 100, then 5 of 56 at 250 and 0 of 56 at 400); with the clipped surrogate and a learning rate of 10^{-5} the run was stable over 400 iterations.

Selection and evaluation. Checkpoints are selected only on validation episodes of the train split. Final numbers use 32 fresh episodes per task template on every test split, from seed blocks disjoint from every episode used for training or selection, with deterministic actions and Wilson 95% intervals [14].

4 Benchmarks

FrankaKitchen. We use the official Gymnasium-Robotics FrankaKitchen environment and the kitchen-complete task of D4RL [3]: open the microwave, move the kettle, flip the light switch and slide the cabinet, in that order. Flat reinforcement learning fails on it [4]. We report the benchmark score (tasks ever completed at the threshold 0.3) from the benchmark start and from starts whose joints are perturbed by 0.2 and 0.3 rad, which were never used in training, and a strict score counting elements that end within 0.1 of their goal.

Articulated multi-step manipulation. The Panda works a tabletop kitchen (Figure 3): it pulls drawers open and pushes them shut, lifts a cabinet lid, and puts 41 scanned household objects into drawers, onto the shelf, into the bin or onto the region, or stacks them into towers. Each skill has a strict physical check; placing, for example, requires 128 support points of the object’s collision hull inside the receptacle, no robot contact, and ten consecutive steps at rest. Seven training templates have three to seven subgoals, and four held-out compositions have five to eight; the horizon is 200 plus 300 control steps per subgoal (1,100 to 2,600 steps). Progress is derived from the scene every step, so undoing a subgoal makes it current again. Five splits are preregistered and checked against the code: train, an iid test split, unseen objects (the 14 held-out objects of a separate grasp catalogue), unseen furniture (every layout factor from a range disjoint from training) and unseen composition. All furniture is built from MuJoCo primitives and resized per episode through per-simulation model fields, so 128 episodes step in parallel. The privileged teacher solves 111 of 112 iid test episodes, 111 of 112 with unseen objects, 110 of 112 with unseen furniture and 61 of 64 unseen compositions.

5 Results

FrankaKitchen. Table 1 reports the official environment over 50 test episodes per condition. The four-task

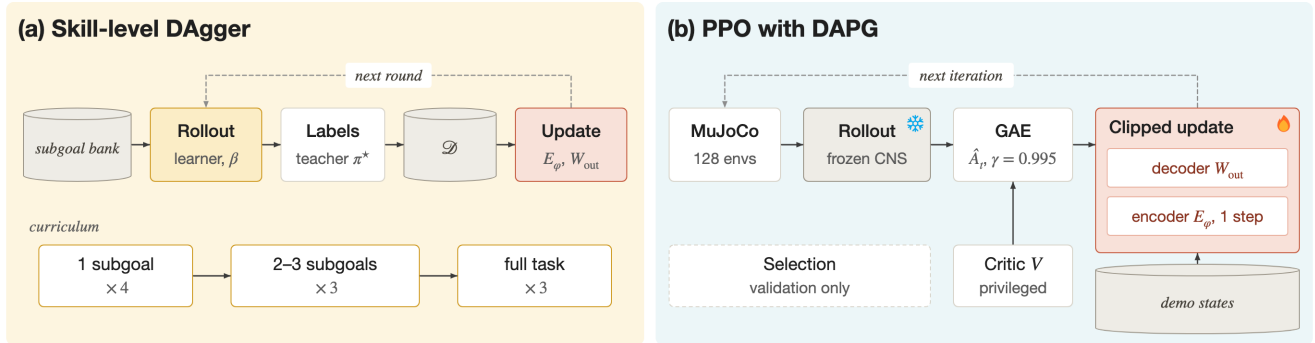


Figure 2: Training on the manipulation benchmark. (a) Skill-level DAGger: each round resets 256 batched episodes to recorded subgoal starts or true starts, rolls out the learner (mixed with the teacher with probability β in the first two rounds), labels every visited state with the privileged scripted teacher π^* , aggregates the labels in $\mathcal{D}$, and updates E_ϕ and W_{out} by truncated backpropagation through the frozen connectome; the curriculum runs four rounds of single subgoals, three of two to three subgoals and three of full tasks. (b) PPO with DAPG: 128 batched environments, rollouts through the frozen connectome, a critic on privileged state and GAE, clipped updates for the decoder and for the encoder through one control step, demonstration terms for both from states replayed through the current encoder every 25 iterations, and checkpoint selection on validation episodes only.

Table 1: FrankaKitchen, official environment, 50 test episodes per condition. Score: tasks completed at the benchmark threshold, times 25. Strict: elements within 0.1 of their goal at the end.

Controller	Start	Score	All four	Strict
Four-task	benchmark	100.0	50/50	50.0
	0.2 rad	100.0	50/50	50.0
	0.3 rad	100.0	50/50	50.0
Motion quality	benchmark	100.0	50/50	69.5
	0.2 rad	97.5	48/50	70.5
	0.3 rad	96.5	46/50	68.0

controller completes all four tasks in every episode, including from perturbed starts it never trained on; the published behaviour-cloning result on kitchen-complete is about 65 [3]. Blind replay of a demonstration from the perturbed starts scores 16 to 19, so the scores measure closed-loop control. The motion-quality controller adds penalties for stray contact, for disturbing other objects and for large and abrupt commands, holds half of each bonus until the element is within 0.1 of its goal, and raises the strict score from 50 to about 70 at a small cost in the benchmark score from perturbed starts. The four-task stage replicates on three PPO seeds (100.0, 100.0 and 97.5 over 20 perturbed test episodes in the batched environment).

Articulated manipulation. Table 2 compares the three stages of the recipe on the same fresh test episodes. PPO on the decoder alone adds 4 to 6 points to imitation; training the encoder as well adds another 13 points on held-in test episodes and 22 points with unseen ob-

Table 2: Articulated manipulation: full-task success from true starts, 32 fresh episodes per template (224 per split, 128 for four held-out compositions), Wilson 95% intervals for the final controller. One training seed.

Split	Imitation	PPO (dec.)	PPO (enc., dec.)
iid	51.3	56.3	69.6 (63–75)
unseen objects	49.6	53.1	75.0 (69–80)
unseen furniture	27.2	32.1	36.6 (31–43)
held-out comp.	18.8	25.0	26.6 (20–35)
subgoals done, iid	68.7	73.2	78.9

jects (episode-level Fisher exact tests against decoder-only PPO: $p = 0.005$ and $p = 2 \times 10^{-6}$). On unseen furniture and unseen compositions the three stages differ little, and the differences are not significant ($p = 0.37$ and 0.89). The final controller completes 79% of the subgoals of iid episodes and 84% with unseen objects, with stray robot contact in about 3% of control steps.

Where it fails. A failure analysis of the decoder-only PPO controller on 16 fresh episodes per template found nearly every failure to be the learner’s (the teacher solves the same episodes): of 218 failed episodes, 151 stopped at a placement, most often on the shelf (44) and in the region (29), then at stacking (10) and closing a drawer (8). Placing into a drawer is solved (16 of 16 on every split), while `retrieve_to_shelf`, a held-out composition, stops at opening the drawer in 16 of 16 episodes. On unseen furniture the controller reaches handles and receptacles at positions outside the training ranges less reliably, which a state-based encoder trained on 1.8 M steps of seen layouts does not extrapolate to.

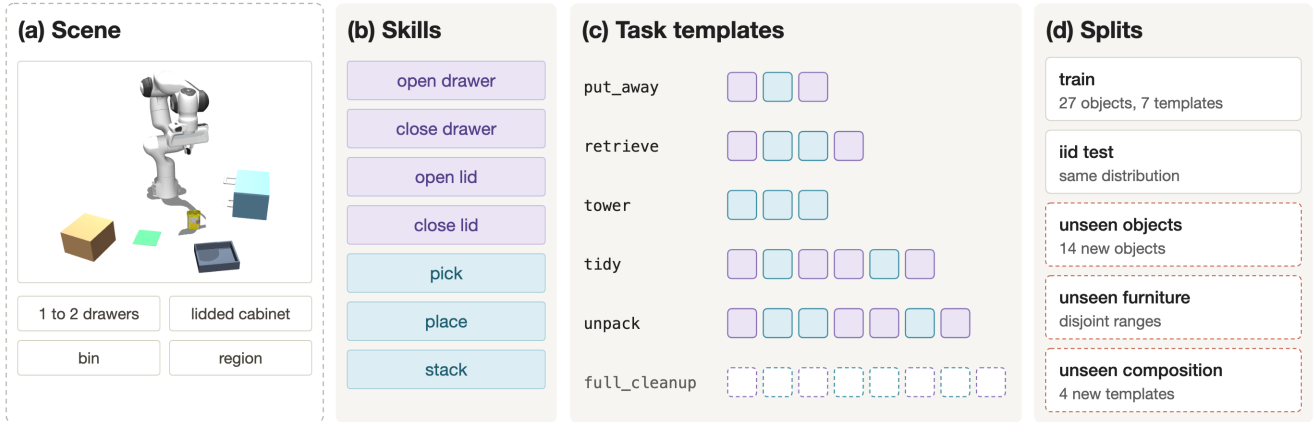


Figure 3: The articulated multi-step manipulation benchmark. (a) A real render of a reset: a drawer cabinet with one or two drawers, a cabinet with a hinged lid over a shelf, a bin and a marked region, with two to four scanned household objects. (b) Seven skills with physical success checks: articulations (purple) and object skills (blue). (c) Task templates as sequences of skills; the four held-out compositions (here `full_cleanup`, dashed) contain adjacent pairs of steps that no training template has. (d) Five preregistered splits; red dashed splits hold out objects, furniture ranges or task orders.

Table 3: Changes that did not improve the controller: successes on the 112-episode validation set (16 per template). Rows marked [†] were scored before the change to capsule rods (Section 5), which moves scores by one or two episodes.

Controller	Validation
Imitation, selected round	64/112
+ failure-targeted resets, best round [†]	64/112
+ gated readout, best round	63/112
PPO, decoder only	75/112
+ failure-targeted resets [†]	73/112
PPO, encoder and decoder, unclipped	83/112
PPO, encoder and decoder, clipped (reported)	86/112

What did not help. Several changes that were expected to help did not (Table 3). Weighting subgoal resets towards the failing placements raised in-run validation but not the 112-episode validation set, both in imitation and in PPO. A readout gated by the observable skill and motor phase, one decoder per program, was motivated by the finding that the teacher is close to linear within a skill and phase (L1 0.068 per skill and phase against 0.258 for one linear map from the observation) but fitted the teacher no better than the single decoder, which already reads out of a trained nonlinear encoder and the connectome, and generalized worse to unseen furniture (5 of 56 against 15 of 56). Separate decoders per program also took full-size Adam steps on rarely visited programs; a shared decoder with slow per-program corrections fixed the instability but not the generalization.

A simulator failure. Two training runs died without an error message. Crash reports showed segmentation faults inside MuJoCo 3.13’s native convex collision (EPA) at the same state twice: a finger pad, a box, pressed 7.9mm into the lid’s handle bar, a cylinder, a pair with no analytic collider. Raising the collision iteration cap only moved the failure; every furniture rod is now a capsule of the same end-to-end length, whose collider against a box is analytic, which changed validation scores by one or two episodes in 112.

6 Controls, limitations and next steps

Controls. The claim of this report is that a frozen whole-CNS connectome can be trained to control the arm on these tasks. Whether its measured wiring matters is a separate claim that needs controls run through the identical pipeline with several seeds: a degree-preserving shuffle of the connectome, parameter-matched GRU and MLP controllers, and lesions. So far only their imitation stage exists on the manipulation benchmark, with one seed each: on iid validation the measured connectome reaches 51.8%, its shuffle 44.6%, a matched GRU 66.1% and a matched MLP 26.8%; the episode-level difference between the measured wiring and its shuffle, pooled over splits, has $p = 0.016$, but seed-level statistics are required before it is a claim, and the GRU, which trains its own recurrence, is ahead. In an earlier real-contact pick-and-place task the complete connectome lifted 58 of 72 test cubes against 27 for its shuffle and 33 for a matched GRU over three training seeds, and resetting the neural state every step removed lifting

entirely, but that task and interface are superseded here.

Limitations. Observations are simulator state, not vision, and on the manipulation benchmark the task cue, including the motor phase, is computed from the task plan and from geometric predicates of the simulator state, so part of the task structure is given to the controller rather than inferred. Removing the phase cue, so that the connectome’s recurrent state has to carry the phase, is running now. The rate model is an abstraction on the measured wiring, not a physiological or spiking emulation. All results are in simulation, and the manipulation results are one training seed so far.

Next steps. Seeds and controls for the reported controller; the no-phase-cue controller; perception from rendered images through a privileged-teacher-to-student distillation; and transfer to public benchmarks.

Reproducibility

Code, configurations, the research log with every experiment including failed ones, and all result files are in the repository. The connectome pack is rebuilt from hash-pinned official exports and fingerprinted. Every figure in this report is generated by a script in `docs/figures/src`.

References

- [1] Stuart Berg et al. Sexual dimorphism in the complete connectome of the *Drosophila* male central nervous system. *bioRxiv*, 2025. doi: 10.1101/2025.10.09.680999. Published in Cell, 2026. Dataset: MaleCNS v1.0, <https://male-cns.janelia.org>.
- [2] Sven Dorkenwald et al. Neuronal wiring diagram of an adult brain. *Nature*, 634:124–138, 2024.
- [3] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4RL: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- [4] Abhishek Gupta, Vikash Kumar, Corey Lynch, Sergey Levine, and Karol Hausman. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2019.
- [5] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- [6] Janne K. Lappalainen et al. Connectome-constrained networks predict neural activity across the fly visual system. *Nature*, 634:1132–1140, 2024.
- [7] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In *Robotics: Science and Systems (RSS)*, 2018.
- [8] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [9] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [11] David Sussillo and L. F. Abbott. Generating coherent patterns of activity from chaotic neural networks. *Neuron*, 63(4):544–557, 2009.
- [12] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.
- [13] Sibor Wang-Chen et al. NeuroMechFly v2: simulating embodied sensorimotor control in adult *Drosophila*. *Nature Methods*, 21:2353–2362, 2024.
- [14] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [15] Kevin Zakka, Yuval Tassa, and MuJoCo Menagerie Contributors. MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo. https://github.com/google-deepmind/mujoco_menagerie, 2022.